

Delegates in C# with Examples

In this article, I am going to discuss the **Delegates in C#** with Examples. Please read our previous article where we discussed the [Roles of Events, Delegates, and Event Handler in C#](#) with Examples. As part of this article, we are going to discuss the following important pointers in detail.

1. [What are Delegates in C#?](#)
2. [How to invoke methods using delegates in C#?](#)
3. [Examples of using Delegates.](#)
4. [Rules of using Delegates in C#.](#)
5. [What are the types of delegates?](#)

What are Delegates in C#?

In simple words, we can say that the delegates in C# are the Type-Safe Function Pointer. It means they hold the reference of a method or function and then call that method for execution. If this is not clear at the moment, then don't worry we will come to this point once we start discussing the examples using delegates.

How to Create a Custom Delegates in C#?

When it comes to creating a custom delegate, is a very simple process. So, a custom delegate can be defined by using the delegate keyword. And it is a kind of magic keyword because behind the scene, when the compiler sees the delegate keyword, it actually generates a class that inherits from some other .NET Framework delegate classes and that will be covered in a moment.

The syntax to create a delegate in C# is very much like the abstract method declaration. In abstract method declaration, we use the abstract keyword whereas, in delegate, we need to use the delegate keyword. The syntax for defining a delegate in C# is as follows:

<Access Modifier> delegate <Return Type> <Delegate Name> (Parameter List);

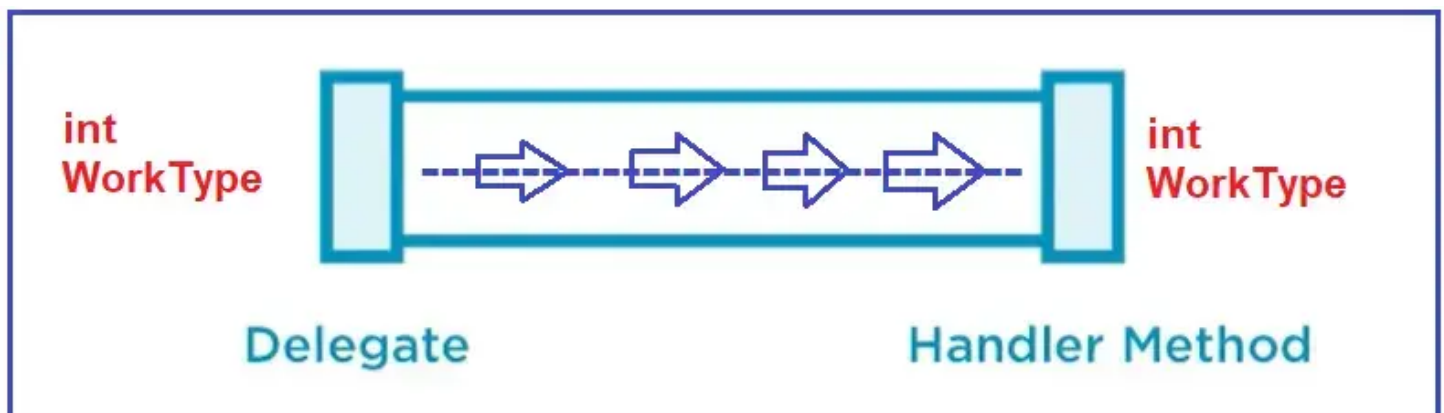
The following is an example of a delegate. Here, you can notice that we are using the delegate keyword. This particular delegate you can think of is a one-way pipeline. It's void, nothing is going to return back. Data only goes forward. The name of the delegate is WorkPerformedHandler and it takes two parameters. The first parameter is integer hours and the second parameter is worktype which is going to be an Enumeration. Now, if you remove the delegate keyword, then it looks like a method.

public delegate void WorkPerformedHandler(int hours, WorkType workType);

In reality, a delegate is a blueprint for the method, which is going to dump the data into the event handler. Visually, it would look like the below image. The delegate is kind of a pipeline. And what we want then, is a way to dump some data from Point A to Point B which is going to be our Event Handler method.



Now, we have our data somewhere in our program and we want to route those data over to this Handler method. How we can route the data into the Handler Method? We are going to route the data which is stored somewhere in our program over to this Handler Method using the pipeline i.e. using a delegate. In delegate, we need to define the parameters which will route the data from point A to point B (i.e. Handler method).



In this case, the pipeline only accepts two parameters and it must be of type int and WorkType otherwise it will not compile. So, now we have one way to delegate the data from point A to Point B. So, the delegate knows how to pass data. So, defining a delegate is very simple as you can see here.

Now, let us try to understand what the Handler method must do in order for this to work. The delegate signature and handler method signature must be matched. As we have defined the delegate with two parameters or you can say our pipeline takes two parameters of int and WorkType type. Now, if the handler method wants to receive the data from the pipeline, then the handler must have the same number, type, and order of parameters as the delegate. For a better understanding, please have a look at the below image which shows the delegate and handler method.

```

//Delegate
public delegate void WorkPerformedHandler(int hours, WorkType workType);

//Handler Method
public void Manager_WorkPerformed(int workHours, WorkType wType)
{
    //Processing the Data
}

```

As you can see in the above image, the first parameter of our delegate is int, and also int is the first parameter of our Handler. The second parameter of the delegate is WorkType and in order to receive the data from the pipeline, the second parameter of the Handler must and should also be WorkType. This is important and parameters type, order, and number must be the same, otherwise, the Handler Method will not receive the data from the pipeline. The parameter names do not matter. You can see that I have given the parameter name of the delegate as hours and workType and I have provided different names to the handler Method and that is fine.

Note: The point that you need to remember while working with C# Delegates is that the signature of the delegate and the method it points should be the same. So, when you create a delegate, then the **Access Modifier, Return Type, and Number, Type, and Order of Parameters** of the delegates must and should be the same as the **Access Modifier, Return Type, and Number, Type, and Order of Parameters** of the function that the delegate wants to refer. You can define the delegates either within a class or just like other types we defined under a namespace.

What happens behind the scene with our delegate?

Now, we are going to discuss what happens behind the scene with our delegate i.e. we are going to discuss the delegate base classes in .NET Framework.

Delegate Base Class in C#:

One of the really core classes in the .NET Framework is Delegate which provides some base functionalities. If you go to the definition of the Delegate class, then you will see that it is an abstract class as shown in the below image.

```

...public abstract class Delegate : ICloneable, ISerializable
{
    ...protected Delegate(object target, string method);
    ...protected Delegate(Type target, string method);

    ...public MethodInfo Method { get; }
    ...public object Target { get; }
}

```

The Delegate class provides two important properties. They are as follows:

1. **public MethodInfo Method {get;}**: This property is used to get the method represented by the delegate. That means it returns a `System.Reflection.MethodInfo` describes the method represented by the delegate. It will throw `MemberAccessException` if the caller does not have access to the method represented by the delegate, for example, if the method is private.
2. **public object Target {get;}**: This property is used to get the class instance on which the current delegate invokes the instance method. That means it returns the object on which the current delegate invokes the instance method if the delegate represents an instance method; null if the delegate represents a static method.

Note: The pipeline has to dump the data somewhere, and the `Method` property defines the name of the method where the data is going to be dumped. And the `Target` would be the object instance where the method lives and null in the case of a static method. If the delegate invokes one or more instance methods, the `Target` property returns the target of the last instance method in the invocation list.

This Delegate abstract class also has one important virtual method called `GetInvocationList`.

1. **public virtual Delegate[] GetInvocationList()**: This method returns the invocation list of the delegate. That means it returns an array of delegates representing the invocation list of the current delegate.

MulticastDelegate Base Class in C#:

Now, let us proceed and understand another important core class i.e. `MulticastDelegate`. If you will go to the definition of `MulticastDelegate` class, then you will see that this class is also an abstract class and this class is inherited from the `Delegate` abstract class as shown in the below image.

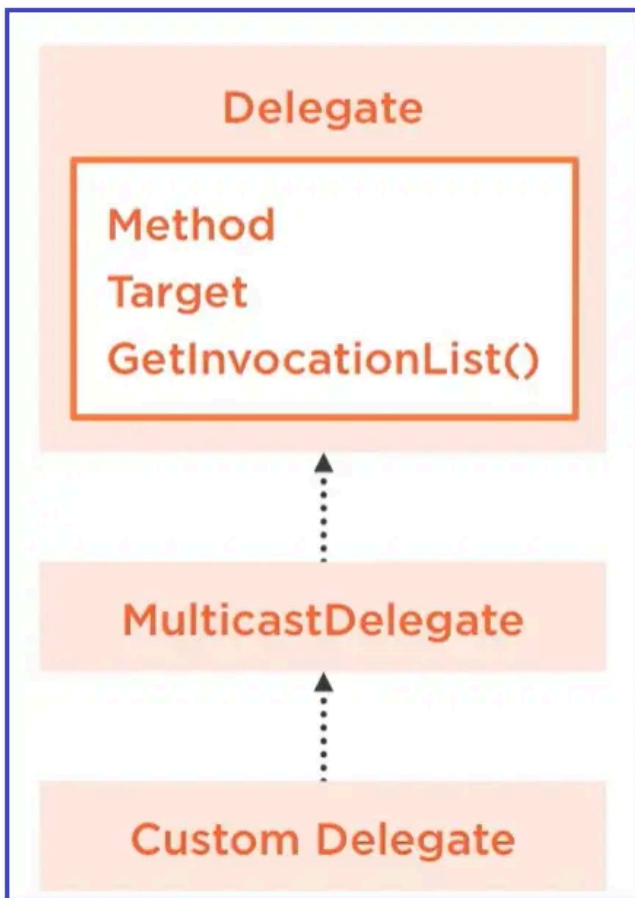
```
...public abstract class MulticastDelegate : Delegate
{
    ...protected MulticastDelegate(object target, string method);
    ...protected MulticastDelegate(Type target, string method);

    ...public sealed override bool Equals(object obj);
    ...public sealed override int GetHashCode();
    ...public sealed override Delegate[] GetInvocationList();
    ...public override void GetObjectData(SerializationInfo info, StreamingContext context);
    ...protected sealed override Delegate CombineImpl(Delegate follow);
    ...protected override MethodInfo GetMethodImpl();
    ...protected sealed override Delegate RemoveImpl(Delegate value);

    ...public static bool operator ==(MulticastDelegate d1, MulticastDelegate d2);
    ...public static bool operator !=(MulticastDelegate d1, MulticastDelegate d2);
}
```

Now, every delegate that we create, once compiled, will inherit from the `MulticastDelegate`. Once we start programming, I will show you this practically by showing the compiled code i.e. IL code by using the `ILDASM` tool.

So, `MulticastDelegate` is a way to hold multiple delegates. For example, I have one message that I want to send over multiple pipelines which will dump the same data into multiple Handler Methods. So, your custom delegate as we discussed earlier will inherit from the `MulticastDelegate`. The complete hierarchy is given below.



Note: The point that you need to remember is, in your code while declaring the delegate, you cannot directly inherit from the `Delegate` or `Multicast Delegate` classes. The way you need to do this is to use the `delegate` keyword and the rest of the things are done by the compiler. These are the special base classes that the compiler restricts us from directly inheriting. Once the compiler sees the `delegate` keyword in the signature, then it will automatically generate the class that inherits from the `Multicast Delegate`.

How to use the Delegate in C#?

How to use the delegate means how we are going to use the delegate to move the data. For this, we need to create an instance of the delegate. And while creating the instance we need to specify the Handler Method name where we want to dump the data. If the Handler Method is a static method, you can access that method directly or using the class name and if the Handler Method is a non-static method, then you need to access the Handler Method using the object name. For a better understanding, please have a look at the below image.

Step1: Declaring the Delegate Signature

```
public delegate void WorkPerformedHandler(int hours, WorkType workType);
```

Step2: Creating the Handler Method with the same signature as Delegate

```
public static void Manager_WorkPerformed(int workHours, WorkType wType)
{
    //Processing the Data
}
```

Step3: Creating an Instance of Delegate

```
WorkPerformedHandler del1 =
    new WorkPerformedHandler(Manager_WorkPerformed);
```

When we declare a delegate, behind the scene when the compiler sees the delegate keyword, it will create a class inheriting from the MulticastDelegate and as this is a class, we can create an instance of the delegate by using the new keyword. And notice to the constructor, that we are passing the Delegate handler Method name. In our example, as the Handler Method is a static method and as both method and the instance that we are creating are present in the same class, we can pass the method name without using the class name, even if you use the class name, then it would not be an issue. But if the method is non-static, then you need to create an instance of the class to which the method belongs, and using that instance, you need to call the method inside the delegate constructor.

How to Invoke a delegate in C#?

Invoking a delegate is very simple. The way we call a method, in the same way, we can call a delegate, and the values for the parameters we need to pass within the parenthesis are as follows. Here, we are passing 5 as the work hours and WorkType as Golf.

```
del1(10, WorkType.Golf);
```

And the above statement will invoke the handler method Manager_WorkPerformed dynamically at runtime. For a better understanding, please have a look at the below image.

```
public delegate void WorkPerformedHandler(int hours, WorkType workType);

WorkPerformedHandler del1 =
    new WorkPerformedHandler(Manager_WorkPerformed);
del1(10, WorkType.Golf);

public static void Manager_WorkPerformed(int workHours, WorkType wType)
{
    //Processing the Data
}
```

Once we create an instance of a delegate, then we need to call the delegate by providing the required values to the parameters so that the methods get executed internally which is bound with the delegates. We can also use Invoke method to execute the delegates. For example:

```
del1.Invoke(10, WorkType.Golf);
```

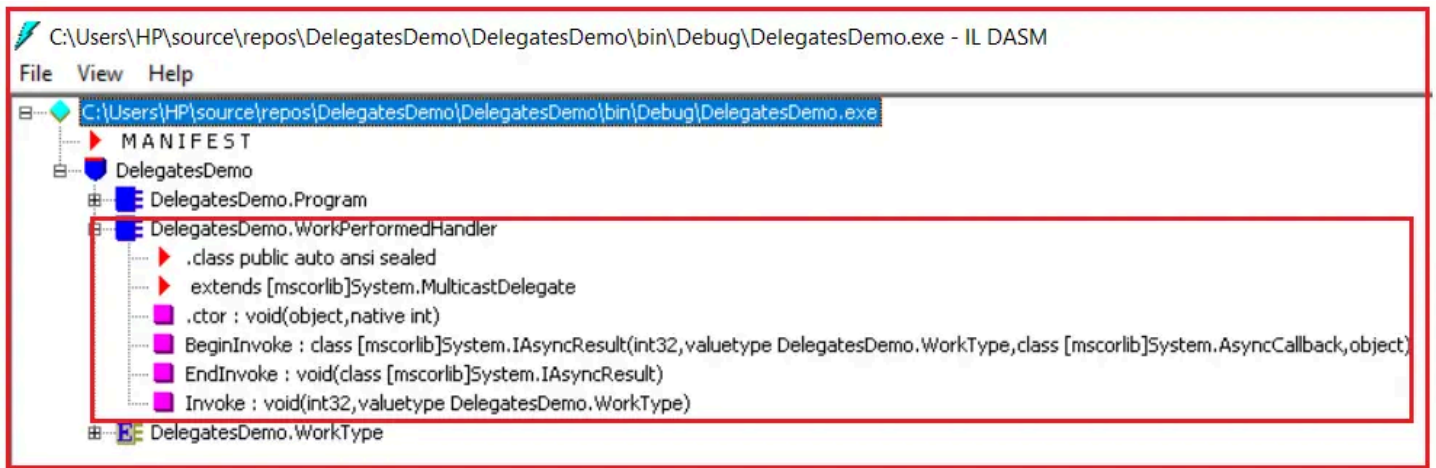
The complete example code is given below.

```
using System;
namespace DelegatesDemo
{
    public delegate void WorkPerformedHandler(int hours, WorkType workType);
    class Program
    {
        static void Main(string[] args)
        {
            WorkPerformedHandler del1 =
                new WorkPerformedHandler(Manager_WorkPerformed);
            del1(10, WorkType.Golf);
            //del1.Invoke(50, WorkType.GotoMeetings);
            Console.ReadKey();
        }
        public static void Manager_WorkPerformed(int workHours, WorkType wType)
        {
            Console.WriteLine("Work Performed by Event Handler");
            Console.WriteLine($"Work Hours: {workHours}, Work Type: {wType}");
        }
    }
    public enum WorkType
    {
        Golf,
        GotoMeetings,
        GenerateReports
    }
}
```

Output:

```
Work Performed by Event Handler
Work Hours: 10, Work Type: Golf
```

That fine. You can see now that the Handler method received the data from the pipeline and then process the data. Now, let us see the IL Code of our delegate using the ILDASM tool and you will see the following code. As you can see in the below code, it is a sealed class that extends from MulticastDelegate class and this class has one constructor.



How to Invoke Methods using Delegates in C#?

If you want to invoke or call a method using delegates, then you need to follow the following three steps.

1. **Declare a Delegate**
2. **Instantiating a Delegate**
3. **Invoking a Delegate**

Another Example to Understand Delegates in C#:

Delegates are used to invoke the call-back functions. What it means is we will invoke one function and we will pass the delegate instance as a parameter to that function and we expect that function to invoke the delegate at some point of time which will invoke the callback method referred by the delegate instance.

As you can see in the below example, we have two methods i.e. DoSomework and CallbackMethod. From our Main method, we want to invoke the DoSomework method but we also want to DoSomework method to invoke one method dynamically at runtime, and that method we will provide at runtime. To do so, we want the DoSomework method accepts the delegate as a parameter and at some point, we need to invoke the delegate within the DoSomework method. Here, we are creating an instance of the delegate within the main method which is referring to the CallbackMethod, and passing that delegate instance as a value to the DoSomework method and at runtime when the DoSomework method invokes the delegate, the method which is pointed by the delegate, in this case, CallbackMethod method is going to be executed.

```
using System;
namespace DelegatesDemo
{
    public delegate void CallbackMethodHandler(string message);
    class Program
    {
        static void Main(string[] args)
        {
            Program obj = new Program();
            CallbackMethodHandler del1 = new CallbackMethodHandler(obj.CallbackMethod);
            //Here, I am calling the DoSomework function and I want the
            //DoSomework function to call the delegate at some point of time
            //which will invoke the CallbackMethod method
        }
    }
}
```

```

        DoSomework(del1);
        Console.ReadKey();
    }
    public static void DoSomework(CallbackMethodHandler del)
    {
        Console.WriteLine("Processing some Task");
        del("Pranaya");
    }
    public void CallbackMethod(string message)
    {
        Console.WriteLine("CallbackMethod Executed");
        Console.WriteLine($"Hello: {message}, Good Morning");
    }
}
}

```

Output:

```

Processing some Task
CallbackMethod Executed
Hello: Pranaya, Good Morning

```

Example to understand Delegate class Important Properties and Method in C#:

At the beginning of this article we have discussed, two important properties i.e. Method and Target, and one important method of the Delegate class i.e. GetInvocationList. Now, let us see the use of these properties and method with an example. In the below example, we have created one delegate and we have created one instance method that is referred by the delegate. Then within the Main method, we create an instance and invoke the properties and method. Here, which method is pointed by the delegate, that method prototype is going to be returned by the Method property, in our example, it will be **Void DoSomework(System.String)**. The Target property will return the fully qualified class name from which the event handler method i.e. SomeMethod belongs which in our example is **DelegatesDemo.SomeClass**. The GetInvocationList method will return the list of delegates that are referred by the delegate and, in this case, only one delegate i.e. **DoSomeMethodHandler**. In our next article, we will understand multicast delegate and, in that case, it is going to return multiple delegates.

```

using System;
using System.Reflection;
namespace DelegatesDemo
{
    public delegate void DoSomeMethodHandler(string message);
    class Program
    {
        static void Main(string[] args)
        {
            SomeClass obj = new SomeClass();
            DoSomeMethodHandler del1 = new DoSomeMethodHandler(obj.DoSomework);
            MethodInfo Method = del1.Method;
            object Target = del1.Target;
            Delegate[] InvocationList = del1.GetInvocationList();
        }
    }
}

```

```

        Console.WriteLine($"Method Property: {Method}");
        Console.WriteLine($"Target Property: {Target}");

        foreach (var item in InvocationList)
        {
            Console.WriteLine($"InvocationList: {item}");
        }

        Console.ReadKey();
    }
}
public class SomeClass
{
    public void DoSomework(string message)
    {
        Console.WriteLine("DoSomework Executed");
        Console.WriteLine($"Hello: {message}, Good Morning");
    }
}
}

```

Output:

```

Method Property: Void DoSomework(System.String)
Target Property: DelegatesDemo.SomeClass
InvocationList: DelegatesDemo.DoSomeMethodHandler

```

Note: If the method is a static method, then the Target property is going to return null.

Rules of using Delegates in C#:

1. A delegate in C# is a user-defined type and hence before invoking a method using a delegate, we must have to define that delegate first.
2. The signature of the delegate must match the signature of the method, the delegate points to otherwise we will get a compiler error. This is the reason why delegates are called type-safe function pointers.

What are the Types of Delegates in C#?

The Delegates in C# are classified into two types as

1. **Single Cast Delegate**
2. **Multicast Delegate**

If a delegate is used for invoking a single method, then it is called a single cast delegate or unicast delegate. In other words, we can say that the delegates that represent only a single function are known as single cast delegates.

If a delegate is used for invoking multiple methods, then it is known as the multicast delegate. Or the delegates that represent more than one function are called Multicast delegates.

Where do we use Delegates in C#?

Delegates are used in the following cases:

1. Event Handlers
2. Callbacks
3. Passing Methods as Method Parameters
4. LINQ
5. Multithreading

How many ways we can call a method in C#?

In C#, we can call a method that is defined in a class in two ways. They are as follows:

1. We can call the method using the object of the class if it is a non-static method or we can call the method through the class name if it is a static method.
2. We can also call a method in C# by using delegates. Calling a C# method using delegate will be faster in execution as compared to the first process i.e. either by using an object or by using the class name.

The example that we discussed in this article is of type Single Cast Delegate because the delegate points to a single function. In the next article, I am going to discuss the [Multicast Delegate in C#](#) with Examples. Here, in this article, I try to explain **Delegates in C#** with examples. I hope you understood the need and use of delegates in C#.